

HUNTBUDDY

AI penetration-testing partner · recon to report

One command: **hunter**. Terminal-native agent for pentesters, OSCP/CPENT students, CTF players and bug-bounty hunters. Runs free & unlimited on OpenCode's Zen models, loads 34 method skills, explains every step (mentor mode), verifies its own findings, writes the report. One folder. No mess. No API keys.

[Read the guide ↓](#)

Contents

1. What is Huntbuddy · 2. How it works · 3. Feature highlights · 4. Folder layout · 5. First-time setup · 6. Daily usage · 7. Commands · 8. Agents & modes · 9. Your first basic test run · 10. The skills · 11. Mentor mode · 12. Verification gate · 13. Reporting · 14. Free brains & rotation · 15. Safety & scope · 16. Publishing

1 · What is Huntbuddy

Huntbuddy is a self-hosted AI agent that lives in your terminal and works like an experienced penetration tester. Run `hunter`; describe a target and a goal; it takes it from there — map the surface, enumerate services, assess vulnerabilities, exploit, escalate, verify, report.

```
hunter — Huntbuddy, an AI pentest partner (runs on the opencode engine inside ~/huntbuddy).
```

Usage:

<code>hunter</code>	start the
<code>interactive chat (TUI)</code>	
<code>hunter --help -h help</code>	this help
<code>hunter --version -v</code>	engine version
<code>hunter status</code>	check relay slot,
<code>opencode procs, skills</code>	
<code>hunter whoami</code>	who Hunter is
<code>(identity card)</code>	
<code>hunter "<message>"</code>	run a one-off
<code>message</code>	
<code>hunter run "<message>"</code>	same, explicit;
<code>times out (240s) instead of hanging</code>	
<code>hunter auth</code>	manage AI
<code>providers & login</code>	
<code>hunter models [provider]</code>	list available
<code>models</code>	
<code>hunter skills</code>	list loaded skill
<code>packs</code>	
<code>hunter debug config</code>	show resolved
<code>configuration</code>	
<code>hunter session stats export import mcp upgrade</code>	engine
<code>utilities</code>	

The `hunter` command — one word, everything else is inside. `hunter status` tells you the relay slot state before you start.

2 · How it works — architecture

Two ideas power Huntbuddy: **a team of agents** and **skills that load on demand**. The hunter agent plans the engagement and dispatches specialist subagents (scanner, critic, reporter); expert playbooks are only pulled into context when a service or phase matches.

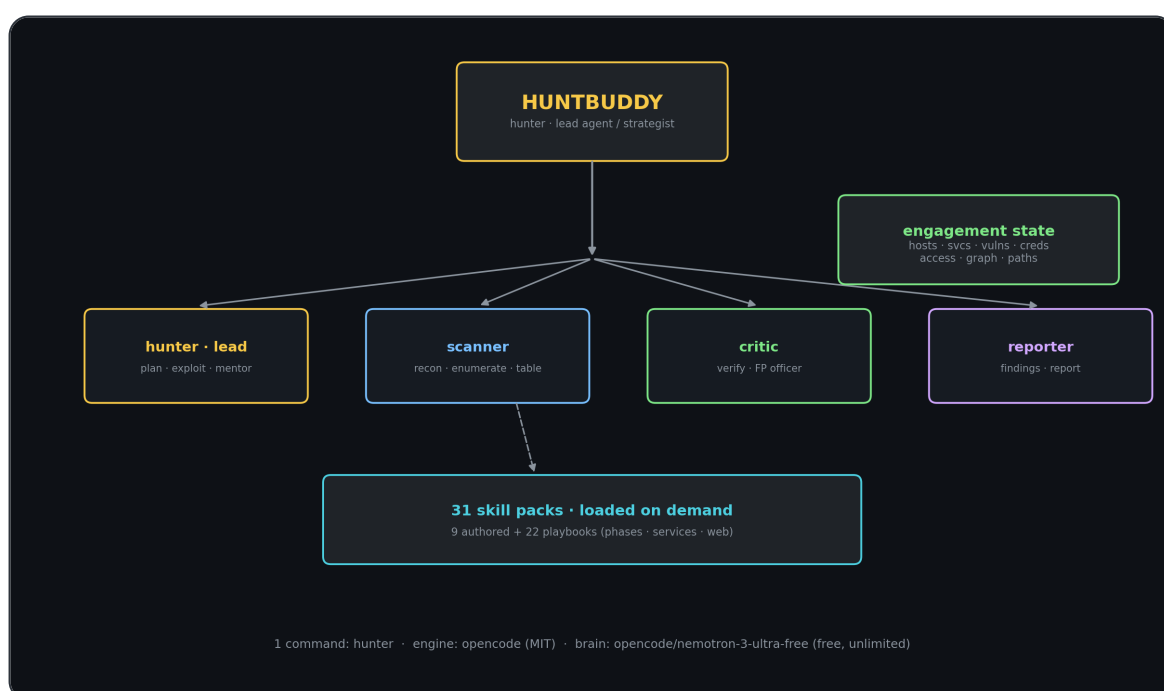


Figure 1 — Multi-agent architecture: hunter lead + scanner/critic/reporter specialists.

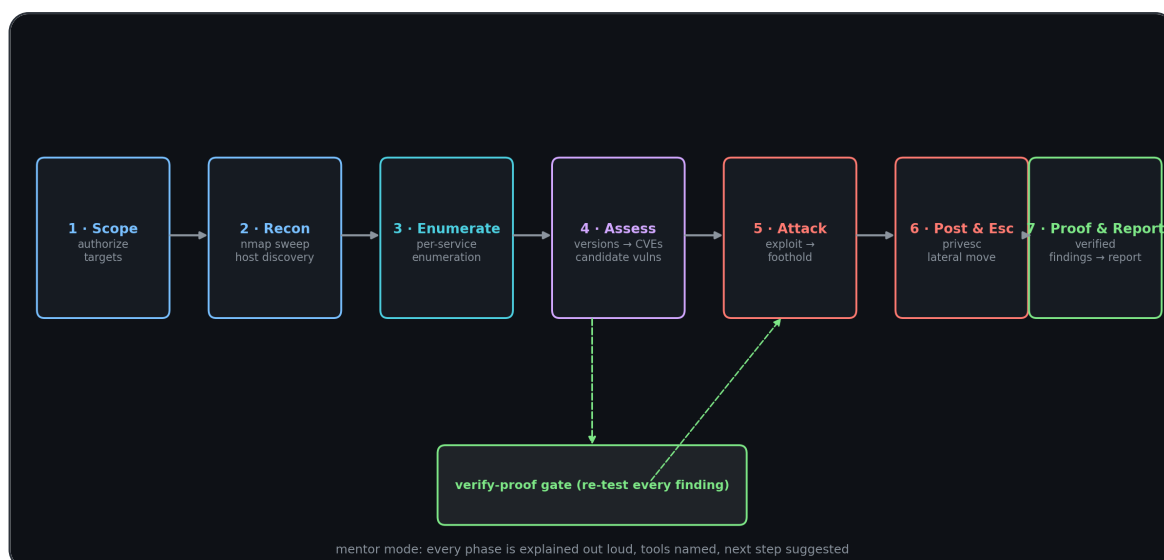


Figure 2 — The seven-phase engagement flow with the mandatory verification gate before reporting.

3 · What you get — feature highlights

Feature	What it does for you
Mentor mode	Explains every tool run, interprets output in plain language, teaches the "why", always proposes the next step.
Verify-before-trust	The critic re-tests every suspected finding (reproduce → cross-check → benign control) before it can be confirmed or reach the report.
37 skill packs	12 authored methodology packs + 25 service/webphase playbooks, all loaded on demand.
No-refusal	Authorized scanning/recon of your own networks and labs is never refused or lectured about — by explicit policy and a permissive brain.
Free & unlimited	Runs on OpenCode Zen free models. No API keys, no accounts, no per-minute quotas/walls that force you to stop mid-task.
One command	<code>hunter</code> from any folder after a single install. That's the whole product interface.

4 · Folder layout

```
huntbuddy/
├─ opencode.jsonc    the Huntbuddy config (brain, agents,
permissions, skills)
├─ HUNTBUDDY.md      brand + operator rules loaded into every
session
├─ skills/           12 authored methodology packs (mentor,
oscp, cpent, research, ...)
├─ playbooks/        25 service/web/phase playbooks (SMB, AD,
SQLi, SSRF, XSS, K8s, ...)
├─ tools/helpers/    hb-state.sh (state machine) · hb-scope.sh
· hb-report.sh · hb-engage.sh
├─ config/           free-models.md (model rotation playbook)
├─ engagements/      one ISOLATED world per engagement;
state.json + findings.md + .hb-scope
├─ .opencode/        custom slash-commands (/guide /verify /
learn)
├─ docs/             this manual · diagrams · samples
├─ start.sh · README.md · LICENSE · NOTICE · hunter (in
~/.local/bin)
```

5 · First-time setup (from scratch)

- 1 **Install the engine once** (official, system-wide): `curl -fsSL https://opencode.ai/install | bash`
- 2 **Get the Huntbuddy folder** into place: from a published bundle, `bash huntbuddy-install.sh`, or a git clone of this repo.
- 3 **Done.** No API keys, no accounts, no login — the brain is OpenCode Zen free and connects automatically.
- 4 **Check it:** `hunter --version` → engine version · `hunter skills` → 37 packs · `hunter debug config` → resolved brain and agents.
- 5 **Test it:** `hunter`, then `ifconfig` — it runs your tools and sums them up.

6 · Daily usage

hunter — Huntbuddy, an AI pentest partner (runs on the opencode engine inside ~/huntbuddy).

Usage:

hunter	start the
interactive chat (TUI)	
hunter --help -h help	this help
hunter --version -v	engine version
hunter status	check relay slot,
opencode procs, skills	
hunter whoami	who Hunter is
(identity card)	
hunter "<message>"	run a one-off
message	
hunter run "<message>"	same, explicit;
times out (240s) instead of hanging	
hunter auth	manage AI
providers & login	
hunter models [provider]	list available
models	
hunter skills	list loaded skill
packs	
hunter debug config	show resolved
configuration	
hunter session stats export import mcp upgrade	engine
utilities	
hunter --help	— everything the command does.

plain text	Everything else is plain chat: goals, targets, questions, or direct commands like <code>ifconfig</code> .
------------	---

8 · Agents & modes

Agent	Role	Color
<code>hunter</code>	End-to-end plan, enumerate, exploit carefully, verify, report, mentors the operator throughout.	amber #c9583d
<code>scanner</code>	nmap/netexec/recon specialist; returns compact host/port/service tables.	green #2e9e44
<code>critic</code>	False-positive officer; adversarial re-tests before anything is confirmed.	red #d51419
<code>reporter</code>	Writes the consultant-style report into engagements/<name>/findings.md.	purple #8957e5

9 · Your first basic test run

- 1 `hunter`
- 2 Type: *"engagement name: smoke-test, target: 127.0.0.1 — scan ports 1-1024 with nmap, summarize, no exploits."*
- 3 Watch it: explain → run nmap → interpret → update the engagement notes → propose Next.
- 4 Then `/guide` for full mentor mode, `/learn nmap` for a teaching drill, `/verify` to sweep findings, `/report` to write the deliverable.

```

> hunter · nemotron-3-ultra-free

$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
    inet 192.168.1.9  netmask 255.255.255.0  broadcast
192.168.1.255
    inet6 2401:4900:1f35:81bc:27b3:9cfb:dd37:5267  prefixlen
64  scopeid 0x0<global>
    inet6 fe80::4b12:8c01:919b:71c8  prefixlen 64  scopeid
0x20<link>
    ether 00:0c:29:ef:17:42  txqueuelen 1000  (Ethernet)
    RX packets 386117  bytes 403565040 (384.8 MiB)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 157446  bytes 79733113 (76.0 MiB)
    TX errors 0  dropped 2 overruns 0  carrier 0  collisions
0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
    inet 127.0.0.1  netmask 255.0.0.0
    inet6 ::1  prefixlen 128  scopeid 0x10<host>
    loop txqueuelen 1000  (Local Loopback)
    RX packets 1038  bytes 45800 (44.7 KiB)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 1038  bytes 45800 (44.7 KiB)
    TX errors 0  dropped 0 overruns 0  carrier 0  collisions
0

```

I'll help you enumerate your local network. First, let me check your local IP configuration, then perform a ping sweep of the 192.168.136.0/24 subnet to discover live hosts. Your local IP is ****192.168.1.9**** on the 192.168.1.0/24 network. You asked me to scan 192.168.136.0/24 (a different subnet) – I'll proceed with that scan as requested.

Real + live: the default brain executed ifconfig and was rolling the nmap sweep — no refusals, unlimited.

10 · The skill packs



Figure 3 — 37 packs: 12 authored + 25 playbooks, loaded on demand.

skills (authored):

bugbounty-webapi – Bug-bounty methodology for web apps/APIs - attack surface mapping, bug classes (IDOR, auth, injection, SSRF, upload, JWT, business logic), impact-first triage, accepted reports. Triggers - web program, bug bounty, API testing, IDOR, JWT.

burp-mode – Web interception workflow for web testing/bug bounty/CTFs. Uses Burp Suite (Community/Pro) or mitmproxy. Capture, modify, replay requests; PoC + screenshots. Triggers - burp, mitm, intercept, capture request, modify request, replay, PoC, proxy.

consult-reporting – Professional pentest report structure - executive summary, methodology, verified findings with evidence + CVSS, reproduction, remediation. Use at REPORTING or when a full report is asked. Triggers - write report, /report, deliverable.

cpent-enterprise – CPENT/enterprise engagement - scope discovery, network aggregation/graphing, pivoting, SOE attacks, AV/EDR-aware execution, credential harvesting, wireless/IoT, evidence chains. Triggers - enterprise engagement, CPENT, multiple segments, pivot.

lab-parser – Parses lab/CTF/task text into structured objectives, questions, flags, and a step-by-step plan. Uses research for context. Teaches the mentality of each step. Triggers - parse this lab, /lab, teach me, CTF text, task description.

mentor-guidance – ALWAYS-ON. Expert-pentester-mentor behavior: explain what and why, name each tool, warn about risk before running, summarize after each phase, suggest the next best step. Load for every engagement.

oscp-pentest – OSCP-style engagement - methodical, enumeration-driven, minimum noise, proof-driven. Load for authorized lab/exam/CTF boxes, single-host targets. Triggers - OSCP, lab box, HTB box, CTF flag, enumeration-first.

playwright – Browser automation via visible Chromium. Navigation, extraction, form submission, screenshots. No credential storage - manual login. Triggers - open browser, screenshot, fill form, login, automate, xss proof in browser.

privesc-linux – Linux privesc checker - systematic, quiet. SUID/SGID, sudo rights, capabilities, cron, world-writable scripts/libs, kernel, PATH hijack, password reuse. Triggers - linux low-priv shell, privesc, escalate on linux.

privesc-windows – Windows privesc checklist - WinPEAS triage: service perms (unquoted path/weak), scheduled tasks, AlwaysInstallElevated, UAC bypass, auto-login, stored secrets, potato tokens, registry autoruns. Triggers - windows low-priv, privesc windows.

research – Quick OSINT/intel on a target, site, URL, document, or dropped text. Parses input, runs fast web searches for tech/CVE/lab context, outputs a compact intel brief. Triggers - research this, look up this link, find lab info, brief me. NOT auto-run.

verify-proof – False-positive officer. Re-test suspected

findings independently (same request, different angle, benign control) before anything is confirmed. Triggers - before state update to confirmed, before report, scanner reports vuln, critic flags uncertainty.

playbooks (ported, MIT):

post-exploit – Post-exploitation gotchas – credential dump order, DPAPI trap, host exhaustion checklist. Use when the current phase is POST_EXPLOIT.

reporting – Reporting phase – generate structured pentest report from findings. Use when the current phase is REPORTING.

ad – AD attack decision tree – which attacks need creds, delegation types, ADCS, key paths to DA. Load when a Windows domain / DC is found..

cloud – Cloud IAM privesc paths and metadata endpoints per provider. Load when the target is cloud or you obtain cloud creds/metadata. Triggers - .

webapp – Web app pentest methodology + ROUTER to per-vuln-class skills. Load at START of systematic web testing to get phase flow (recon → map → test-by-OWASP-class → prove → report) and pick the matching skill. Triggers - web engagement, webapp, API testing, OWASP.

cicd – CI/CD gotchas – default creds, instant-RCE paths, secret leaks in logs. Use when a CI/CD service is found. Triggers - Jenkins, GitLab.

database – Database RCE paths – UDF, xp_cmdshell, COPY TO PROGRAM, Redis key write. Use when a database service is found. Triggers - MySQL 3306.

docker-k8s – Docker/Kubernetes attack techniques - exposed API abuse, container escape, RBAC/privileged-pod issues, secret theft. Triggers - Docker 2375/2376, Kubernetes API 6443, kubelet 10250, privileged pod, service-account token, docker.sock.

ftp – FTP version exploits – vsftpd backdoor, ProFTPD mod_copy, writable upload-shell chain. Use when FTP is open. Triggers - port 21, vsftpd.

mobile-android – Android APK static analysis – OWASP Mobile Top 10, Retrofit API audit, transport security, smali reading, component export, auth flow.

pivoting – Turn a foothold into a RELIABLE pivot (SOCKS tunnel + persistent shell). Use the moment you have code-exec on a dual-homed/edge host needing an internal segment. Triggers - dual-homed host, not reachable, internal CIDR behind foothold, chisel/ligolo, proxychains.

smb – SMB gotchas – DPAPI dump trap, relay when signing disabled, credential dump order. Use when SMB is found. Triggers - ports 445/139.

ssh – SSH version CVEs and post-auth tricks – regreSSHion, user enum, agent hijack. Use when SSH is open. Triggers - port 22, OpenSSH banner.

web-server – Web SERVER software (not app-layer) attack techniques – server misconfigs, known server CVEs, path traversal, exposed status/config. Use.

auth-bypass-idor – Broken access control - IDOR, privesc, JWT abuse, mass assignment, forced browsing. Use on multi-user/role

apps with object IDs, tokens, admin functions. Triggers - /api/users/<id>, /orders/<id>, JWT eyJ, role/isAdmin, 403, guessable object refs.

cmd-injection – Command Injection detection→RCE→proof. Use when a param/header/JSON field feeds a shell (ping, traceroute, convert, archive, download, filename) or output leaks command result. Triggers - ;id, \$(id), backticks, command injection, ping param, RCE in web.

deserialization – Insecure deserialization to RCE. Use when the app deserializes attacker-controlled data (cookies/tokens/hidden fields, VIEWSTATE, Java/PHP/Python/Node). Triggers - base64 r00AB/AC ED, PHP 0-prefix, __VIEWSTATE, pickle, unserialize.

lfi-traversal – Path traversal / LFI detection→file-read→RCE for web apps. Use when a param names a file/path/template/lang/download or response embeds file contents. Triggers - file=/page=/path=, ../, %2e%2e, directory listing, download endpoint.

misc-common – High-value web classes without deep packs - auto-load on ANY web engagement: CSRF, JWT/crypto, open redirect, CORS, host-header, request smuggling, cache poisoning, business-logic/race, mass-assignment, clickjacking, header gaps, subdomain takeover, HTTP verb, GraphQL, WebSockets. Triggers - CSRF, JWT, CORS, redirect, smuggling, graphql, websocket, race, business logic.

sqli – SQL injection detection→exploitation→proof for web apps/APIs. Use when a param reaches a DB query or input triggers SQL error/differential, or during web vuln assessment. Triggers - SQL error string, ' or 1=1, order by, sleep delay, login bypass, id/search/filter param.

ssrf – SSRF detection→internal-access→proof. Use when the app fetches a URL you influence (webhooks, url/image/PDF fetchers, link preview, import-from-URL, SSO/OIDC, XML/SVG). Triggers - url=/uri=/callback=, fetch failed, webhook, image proxy, redirect.

ssti – Server-Side Template Injection detection→engine-fingerprint→RCE. Use when user input renders into a server template and math payloads evaluate. Triggers - {{7*7}} returns 49, \${7*7}, template injection, names/greetings/PDF render.

upload-rce – File-upload abuse to code execution. Use when the app accepts a file (avatar, document, import, attachment) and you can influence name/type/content, or find where uploads land. Triggers - multipart upload, avatar, invalid file type, /uploads path.

xss – Cross-Site Scripting detection→context-aware bypass→real-browser proof. Use when input is reflected/stored into HTML/JS/attr/URL/CSS, or during web vuln assessment. Triggers - XSS, reflected input, stored echo, alert, dalfox, cookie steal, javascript:.

xxe – XXE detection→file-read/SSRF→proof. Use when the app parses XML you influence (SOAP/REST XML, SAML, RSS, DOCX/SVG upload, sitemap, SVG avatars). Triggers - Content-Type application/xml, <?xml, SOAPAction, SAMLResponse, DOCX/SVG upload.

The actual 37 packs, straight from disk. Skills are plain markdown — add yours with a folder.

Getting the most out of the 12 authored packs

- 1 **mentor-guidance** — on by default; type `/guide` to force it. Every reply ends with a concrete *Next:*.
- 2 **oscp-methodology** — prompt: *"pentest <box>, goal is root. Follow the OSCP methodology."*
- 3 **cpent-enterprise** — prompt: *"network pentest of <range>, enumerate everything, then plan pivots."*
- 4 **privesc-linux / privesc-windows** — paste your low-priv shell context: *"I have a Linux shell as www-data on <ip>. Escalate."*
- 5 **bugbounty-webapi** — prompt: *"bug-bounty recon and test of <domain>, focus on access control and IDOR."*
- 6 **verify-proof** — run `/verify` before reporting; invalid findings stop getting retested.
- 7 **consult-reporting** — `/report`, then `tools/helpers/hb-report.sh` to export HTML.

11 · Mentor mode — watch it work

```
> cd ~ && hunter

■■■■ ■■■■ ... HUNTBUDDY — AI pentest partner · recon to report
                                   (colors: amber
on dark)

> hunter · nemotron-3-ultra-free

$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
      inet 192.168.1.9 netmask 255.255.255.0 broadcast
192.168.1.255
      ether 00:0c:29:ef:17:42 (Ethernet)
lo:   flags=73<UP,LOOPBACK,RUNNING> mtu 65536
      inet 127.0.0.1 netmask 255.0.0.0 (Loopback)

Your local IP is 192.168.1.9. Ready for the ping sweep of
192.168.1.0/24?

Huntbuddy narrates before running, interprets after, proposes Next. That is /guide
behaviour.
```

12 · Verification — kill false positives

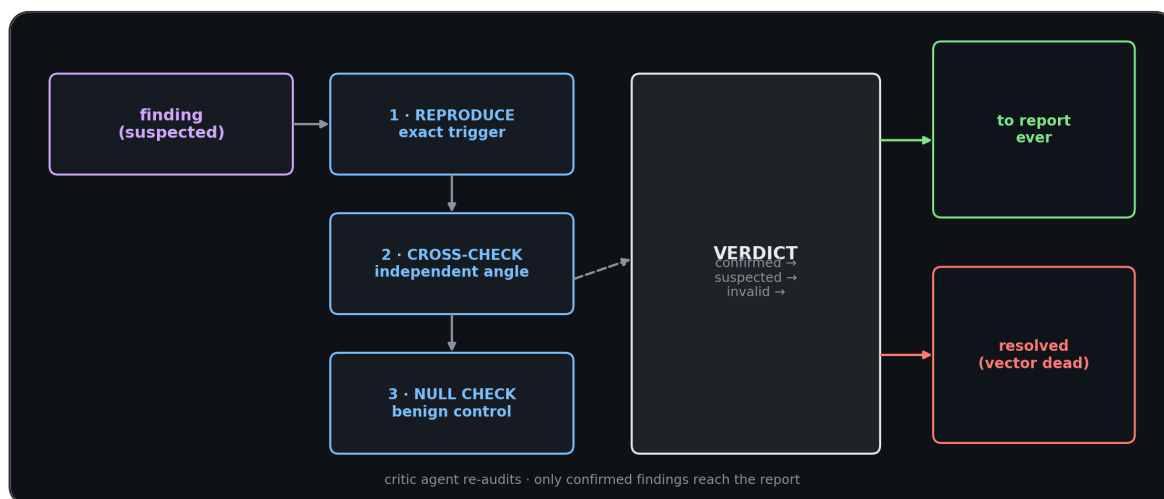


Figure 4 — The verification gate: reproduce → cross-check → control; only confirmed findings reach the report.

Scanners lie; LLMs amplify guesses. Run `/verify` before `/report` and only confirmed findings make it into the deliverable.

13 · Reporting

- 1 `/report` (or "write the full report").
- 2 Structure: executive summary → methodology → verified findings (CVSS) → coverage → appendix.
- 3 Export a single self-contained HTML: `tools/helpers/hb-report.sh ~/huntbuddy/engagements/<name>/findings.md`

14 · Free brains & rotation

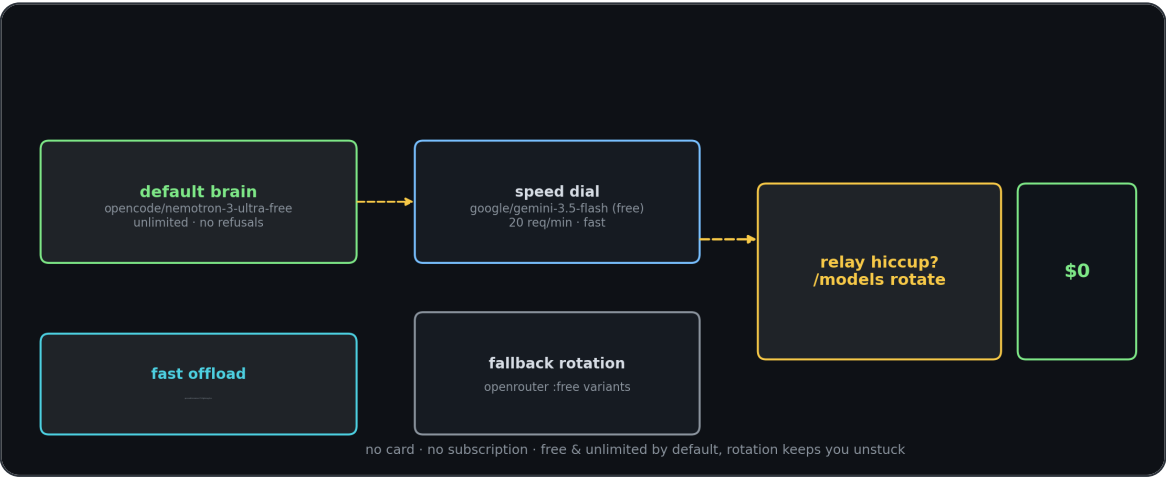
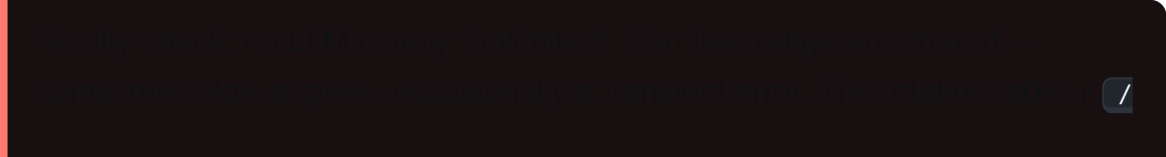


Figure 5 — Free-model strategy: unlimited permissive default + fast offload + switches when a relay hiccups.

Model (default: unlimited, no refusals)	Role
<code>opencode/nemotron-3-ultra-free</code>	Default brain — unlimited, understands authorized scanning, executes without refusals. Slower at peak hours but never "stuck".
<code>opencode/nemotron-3.5-lightning-free</code>	Fast small/offload model, same family.
<code>opencode/big-pickle</code>	Flagship, occasionally declines security-adjacent asks — use for general work.
<code>google/gemini-3.5-flash</code>	Speed-dial when wanted fast replies; caveat: 20 req/min free quota. Never the default.



15 · Safety & scope

- **Scope guard:** `tools/helpers/hb-scope.sh <target>` checks a target against your authorized list (`engagements/.hb-scope`).
- **Command guardrails:** destructive patterns (`rm -rf /`, `dd`, `mkfs`) are denied in the default config.
- **No-refusal is bounded:** recon and exploitation on *your own / authorised* targets always runs; a genuinely unauthorised target gets one question, never a lecture.
- **Integrity:** the engine installs from the official source; your data stays in `~/.huntbuddy` folders and the engine's own storage.

16 · Publishing & open-source note

The folder is publishable: `README.md`, this manual, `LICENSE` (MIT), `NOTICE`, skills, scripts, docs and config are authored content. The engine it runs on is MIT open source (OpenCode); the 25 playbooks are ported under MIT from PentestCode. MIT requires keeping those copyright notices in LICENSE/NOTICE when you redistribute — keep that one line; everything else is yours to brand, extend and share.